

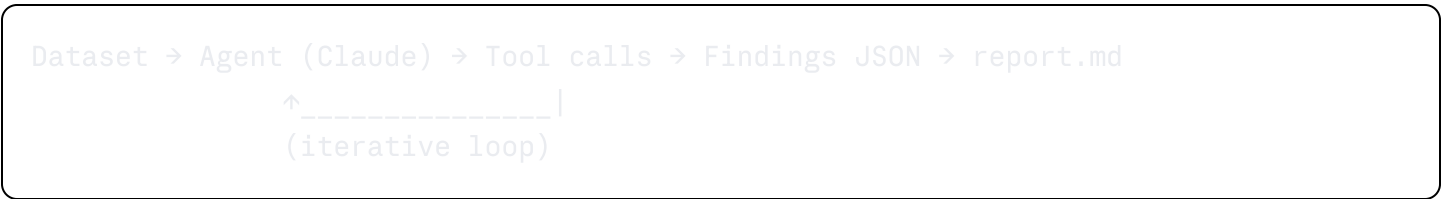
Data Quality Monitoring Agent

An agentic data quality tool powered by Claude (Anthropic API). The agent autonomously investigates a dataset for anomalies, missing values, duplicates, and out-of-range values, then generates a structured Markdown report — no manual configuration required.

How It Works

The agent follows an **agentic loop**:

1. Receives the dataset schema and a sample of rows
2. Decides which tools to call (column stats, anomaly flagging, duplicate checks)
3. Iterates — using each tool result to inform the next investigation step
4. Calls `finish` when satisfied, returning structured JSON findings
5. Renders findings into a clean Markdown report



Detectable Issues

Issue Type	Description
Null values	Missing / NaN entries per column
Negative values	Values below zero in numeric columns
Z-score outliers	Values > 3 standard deviations from the mean
Out-of-range	Values outside a logical domain (e.g. discount > 1.0)
Duplicates	Exact duplicate rows across the dataset

Quickstart

```
bash
```

```
# 1. Clone / copy this project
cd dq-agent

# 2. Install dependencies
pip install -r requirements.txt

# 3. Set your Anthropic API key
export ANTHROPIC_API_KEY=sk-ant-...

# 4a. Run on the built-in sample dataset
python agent.py

# 4b. Run on your own CSV
python agent.py path/to/your_data.csv output_report.md
```

The report is saved as `report.md` by default.

Project Structure

```
dq-agent/
├── agent.py          # Agent loop, tools, and report renderer
├── requirements.txt  # Python dependencies
└── README.md        # This file
```

Swapping in Your Own Data

Pass any CSV file as the first argument:

```
bash

python agent.py my_dataset.csv my_report.md
```

The agent will automatically adapt its investigation to your column names and data types.

Extending the Agent

Add a new rule — extend `flag_anomalies()` in `agent.py` with a new rule branch, then add it to the `enum` in the tool schema.

Add a new tool — define the function, add its schema to `TOOLS`, and add a dispatch case in `dispatch_tool()`. The agent will discover and use it automatically.

Plug in a real database — replace the `pd.read_csv()` call in `main()` with a SQLAlchemy or psycopg2 connection to point the agent at a live Postgres / Snowflake table.

Output Example

```
# Data Quality Report - sample_transactions.csv
_Generated: 2025-03-25 14:02_

## Overview
| Field                | Value                |
|-----|-----|
| Rows                 | 205                  |
| Columns              | 8                    |
| Overall Severity    | ● HIGH |

> Dataset contains negative unit prices, impossible discount values above 1.0,
> missing customer IDs, quantity outliers, and 5 duplicate rows.

## Findings
### 1. Negative Values - `unit_price`
- Affected rows: 6
...
```